KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 04

Laboratuvar Föyü

Kapsülleme onarımı · doğrulamalı setter · salt-okunur alan · değişmez sınıf · paket düzeni

ÖZET

Bu laboratuvar oturumunda önce kapsülleme ihlali içeren bir sınıfı onaracak, ardından iki alan arasında **tutarlılık** kuralı işleten bir setter yazacak, sınıfın kendi ürettiği **salt-okunur** bir kimlik alanı tasarlayacak, dört kurala uyan bir **değişmez (immutable)** sınıf yazacak ve son olarak paket-özel erişimi kullanan iki paketli mini bir kütüphane düzeni kuracaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 4 ders notu Bölüm 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta04_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- Alanları `public` olan bir sınıfı, `private` alan + denetimli kapı düzenine dönüştürebilir.
- Geçersiz değer geldiğinde nesnenin **eski geçerli durumunu koruduğunu** gösteren setter yazabilir.
- İki alan arasında **tutarlılık** gerektiren bir kuralı tek bir metotta uygulayabilir.
- `private final` alanla salt-okunur bir kimlik tasarlayabilir ve değerini sınıfın kendisine ürettirebilir.
- Değişmez (immutable) bir sınıfı dört kuralına uygun yazabilir ve dönüşüm metotlarının yeni nesne döndürdüğünü gösterebilir.
- Paket bildirimi yazabilir, paket-özel (varsayılan) erişimi bilinçli olarak kullanabilir.
- Bir alan için getter/setter yazılıp yazılmayacağına gerekçeli karar verebilir.



Şekil 4.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

1. Her sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.

2. Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`
3. Sınıf adları `PascalCase`, değişken ve metot adları `camelCase`, sabitler `SCREAMING_SNAKE_CASE` olmalıdır.
4. Değişken, sınıf, metot ve paket adlarında **Türkçe karakter kullanmayın**.
5. **Tüm alanları `private` yapın.** Tek istisna `public static final` sabitlerdir.
6. Bir setter yazmadan önce ders notundaki **Şekil 4.8**'deki karar akışını uygulayın: dışarısı bu alanı yazmak zorunda mı, yoksa değişim bir **iş olayı** mı?
7. Geçersiz istekleri sessizce yutmayın; ekrana `RED:` ile başlayan bir uyarı yazın ve **atamayı yapmayın**.
8. Her sınıfta `toString()` metodunu ezin; sürücü programlarda nesneleri bu metotla yazdırın.

> **DİKKAT**
>
> Teslim edilen kod **bireysel çalışmanın** ürünü olmalıdır. Arkadaşınıza kod açıklamak serbesttir; kod kopyalamak intihaldir. Kaynak dosyalar benzerlik denetiminden geçirilir. İnternetten ya da yapay zekâ araçlarından alınan kod, kaynağı belirtilmeden teslim edilirse değerlendirmeye alınmaz.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre	Puan	Zorluk
10 dakika	Puansız — **zorunlu**	●○○

Amaç

Java ortamının çalıştığını doğrulamak ve `private` alan + denetimli metot düzenini hatırlamak.

Görev Tanımı

`Gorev0_Kontrol` adında bir sınıf yazın. Aynı Java dosyasında, ayrı bir `Sayac` sınıfı tanımlayın. `Sayac` sınıfının `deger` alanı `private` olmalı ve yalnızca `artir()` metoduyla değişebilmelidir; `setDeger()` **yazmayın**.

Gereksinimler

1. `Sayac` sınıfının `etiket` alanı `private final` olmalı, yapıcıda atanmalıdır.
2. `deger` alanı `private` olmalı, yalnızca `artir()` ile artmalıdır.
3. `getDeger()` getter'ı yazılmalı, `setDeger()` **yazılmamalıdır**.
4. `toString()` ezilmelidir.
5. `main` Java sürümünü yazdırmalı, üç kez `artir()` çağırıp sonucu göstermelidir.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev0_Kontrol
Java surumu : 25.0.2
Calisma dizini hazir.
Sayac[lab04=3]
Deger yalnizca artir() ile degisti: 3
ORTAM HAZIR.
```

> **İPUCU**
>
> Java sürümünü `System.getProperty("java.version")` ile okuyabilirsiniz. Sürüm numaranız farklı olabilir; bu bir sorun değildir.

Kontrol Soruları

1. `deger` alanı için neden `setDeger()` yazılmadı? Bu kararın gerekçesini bir cümleyle yazın.

2. `etiket` alanını `final` yapmak neyi engeller?

Görev 1 — Kapsülleme İhlalini Onarma

Süre	Puan	Zorluk
25 dakika	20 puan	●○○

Amaç

Alanları `public` olan bir sınıfı, veriyi koruyan ve kuralı içinde barındıran bir sınıfa dönüştürmek.

Görev Tanımı

Aşağıdaki bozuk `Urun` sınıfı size verilmiştir. Bu sınıfı kapsülleme ilkesine uygun hâle getirin ve `Gorev1_UrunDemo` adında bir sürücü program yazarak davranışını gösterin.

Urun.java

```java
// BOZUK SURUM - Gorev 1'de bunu onaracaksiniz
public class Urun {
  public String barkod;       // kimlik disaridan degistirilebiliyor
  public String ad;           // bos birakilabiliyor
  public double fiyat;        // negatif olabiliyor
  public int stok;            // eksiye dusebiliyor

  public Urun(String barkod, String ad, double fiyat, int stok){
    this.barkod = barkod;
    this.ad = ad;
    this.fiyat = fiyat;
    this.stok = stok;
  }
}

// Cagiran kod su anda boyle davranabiliyor:
//   u.fiyat = -10.0;      // negatif fiyat
//   u.ad = "";            // isimsiz urun
//   u.stok = -500;        // eksi stok
//   u.barkod = "0";       // kimlik degisti
```

Kod 4.1 — Onarılacak sınıf (verilen hâl)

Gereksinimler

Alan / işlem	Beklenen tasarım
`barkod`	`private final` · yalnızca getter (salt-okunur)
`ad`	`private` · getter + setter; **boş ya da yalnızca boşluk olamaz**

`fiyat`	`private` · getter + setter; **negatif olamaz**
`stok`	`private` · yalnızca getter; `setStok()` **yazılmayacak**
Stok artışı	`stokGir(int adet)` — adet pozitif olmalı
Stok azalışı	`stokCik(int adet)` — adet pozitif ve stoktan küçük/eşit olmalı
Türetilmiş bilgi	`getStokDegeri()` — `fiyat * stok`, **alan tutulmaz**
Geçersiz istek	`RED:` ile başlayan uyarı yazılır, atama yapılmaz
`toString()`	Ezilir; barkod, ad, fiyat ve stok görünür

Beklenen Çıktı

```
▸ TERMINAL — java Gorev1_UrunDemo
Baslangic : Urun[8690001, Defter A5, 42.50 TL, stok=30]

--- Gecersiz istekler ---
  RED: fiyat negatif olamaz (-10.00).
  RED: urun adi bos olamaz.
  RED: stok yetersiz (500 > 30).
Degismedi : Urun[8690001, Defter A5, 42.50 TL, stok=30]

--- Gecerli islemler ---
Son durum : Urun[8690001, Defter A5, 45.00 TL, stok=23]
Stok degeri: 1035.00 TL
```

> **DİKKAT**
>
> `stok` alanı için setter yazmayınız. Stok, dışarıdan atanan bir sayı değil; **giriş ve çıkış olaylarının** sonucudur. Bu, ders notu Bölüm 4.5'teki “setter mi, iş metodu mu?” ayrımının doğrudan uygulamasıdır.

> **İPUCU**
>
> Yapıcıda alanlara doğrudan atama yapmayın; `setAd()`, `setFiyat()` ve `stokGir()` metotlarını çağırın. Böylece doğrulama kuralını iki kez yazmamış olursunuz. `ad` alanının yapıcıdan önce bir başlangıç değeri alması gerektiğini unutmayın.

Kontrol Soruları

1. `barkod` alanını `final` yapmak yeterli miydi, `private` da gerekli miydi? İkisinin ayrı ayrı ne sağladığını yazın.
2. `getStokDegeri()` yerine `private double stokDegeri;` alanı tutsaydınız hangi sorun ortaya çıkardı? Somut bir çağrı dizisiyle gösterin.
3. `stokCik(500)` çağrısından sonra `stok` alanının değeri neden değişmedi?

Görev 2 — İki Alan Arasında Tutarlılık Kuralı

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Tek bir alana bakarak doğrulanamayan, **iki alanın birlikte** sağlaması gereken bir kuralı koda taşımak.

Görev Tanımı

`KanBasinci` adında bir sınıf yazın. Sınıf, büyük (sistolik) ve küçük (diyastolik) tansiyon değerlerini tutar. Fizyolojik olarak **büyük değer her zaman küçük değerden büyük olmalıdır**. Bu kural tek bir alana bakarak denetlenemez; bu yüzden iki değeri **birlikte** alan bir metot yazacaksınız.

Gereksinimler

1. `buyuk` ve `kucuk` alanları `private int` olmalıdır.
2. `EN_DUSUK = 40` ve `EN_YUKSEK = 250` sabitleri `public static final int` olarak tanımlanmalıdır.
3. `setBuyuk()` ve `setKucuk()` metotları **yazılmayacaktır**. Bunun yerine tek bir `boolean ayarla(int yeniBuyuk, int yeniKucuk)` metodu olacaktır.
4. `ayarla()` üç kuralı denetler: (a) her iki değer de 40–250 aralığında olmalı, (b) büyük değer küçük değerden **kesinlikle büyük** olmalı. Kural ihlalinde `RED:` uyarısı yazılır, **hiçbir alan değişmez** ve `false` döner.
5. `getNabizBasinci()` metodu `buyuk - kucuk` değerini döndürür (türetilmiş).
6. `degerlendir()` metodu `NORMAL`, `YUKSEK-NORMAL`, `EVRE-1` veya `EVRE-2` döndürür.
7. `toString()` `118/76 mmHg (NORMAL)` biçiminde yazdırmalıdır.
8. Yapıcı, alanlara doğrudan atama yapmak yerine `ayarla()` metodunu çağırmalıdır.

Koşul	Değerlendirme
`buyuk < 120` ve `kucuk < 80`	`NORMAL`
`buyuk < 130` ve `kucuk < 80`	`YUKSEK-NORMAL`
`buyuk < 140` veya `kucuk < 90`	`EVRE-1`
Diğer durumlar	`EVRE-2`

Tablo 4.6 — `degerlendir()` metodunun sınıflandırma tablosu

Beklenen Çıktı

```
▸ TERMINAL — java Gorev2_KanBasinciDemo
```

```
Olcum 1 : 118/76 mmHg (NORMAL)
Nabiz basinci: 42

--- Tutarsiz istekler ---
  RED: buyuk tansiyon kucukten buyuk olmalidir (90 <= 130).
  RED: degerler 40..250 araliginda olmalidir.
  RED: buyuk tansiyon kucukten buyuk olmalidir (120 <= 120).
Degismedi: 118/76 mmHg (NORMAL)

--- Gecerli olcumler ---
Olcum 2 : 135/85 mmHg (EVRE-1)
Olcum 3 : 152/96 mmHg (EVRE-2)
Nabiz basinci: 56
```

SIK YAPILAN HATA

En sık yapılan hata, `setBuyuk()` ve `setKucuk()` yazıp her birinde ayrı ayrı denetim yapmaktır. Bu **çalışmaz**: `120/80` durumunda önce `setKucuk(130)` çağrılırsa geçici olarak `120/130` gibi geçersiz bir ara duruma düşülür. Tutarlılık kuralı gerektiren alanlar **birlikte** atanmalıdır.

İPUCU

Yapıcıda önce alanlara geçerli bir başlangıç değeri (`120/80`) verin, sonra `ayarla()` çağırın. Böylece yapıcıya geçersiz değer gelse bile nesne geçerli bir durumda doğar.

Kontrol Soruları

1. Neden `setBuyuk()` ve `setKucuk()` yerine tek bir `ayarla()` metodu yazıldı? Yanıtınızı “geçersiz ara durum” kavramını kullanarak açıklayın.
2. `ayarla()` metodunun `boolean` döndürmesi çağıran koda ne kazandırır?
3. `araliktaMi()` yardımcı metodunu neden `private` yaptınız?

Görev 3 — Salt-Okunur Kimlik ve Otomatik Numara

Süre	Puan	Zorluk
20 dakika	20 puan	●●○

Amaç

Değeri **dışarıdan alınmayan**, sınıfın kendisi tarafından üretilen ve bir daha değiştirilemeyen bir alan tasarlamak.

Görev Tanımı

`Uye` adında bir sınıf yazın. Her üyeye, oluşturulduğu anda otomatik olarak `U-<yil>-<4 haneli sira>` biçiminde bir üye numarası verilmelidir. Bu numara dışarıdan hiçbir biçimde değiştirilememelidir.

Gereksinimler

Alan / metot	Beklenen tasarım
`uyeNo`	`private final String` · yalnızca getter · **yapıcıda üretilir**
`katilimYili`	`private final String` · yalnızca getter
`ad`	`private` · getter + setter; boş olamaz
`puan`	`private int` · yalnızca getter; `setPuan()` **yazılmayacak**
`sonrakiSira`	`private static int` · sınıf düzeyinde sayaç, 1'den başlar
`puanEkle(int miktar)`	Pozitif miktar ekler; değilse `RED:` yazar
`puanHarca(int miktar)`	Pozitif ve mevcut puandan küçük/eşit olmalı
`getSeviye()`	≥1000 `PLATIN` · ≥500 `ALTIN` · ≥100 `GUMUS` · diğer `STANDART` (türetilmiş)
`toString()`	`Uye[U-2026-0001, Mert Aydin, puan=0, STANDART]`

Beklenen Çıktı

```
▸ TERMINAL — java Gorev3_UyeDemo
  RED: uye adi bos olamaz.
a : Uye[U-2026-0001, Mert Aydin, puan=0, STANDART]
b : Uye[U-2026-0002, Zeynep Sari, puan=0, STANDART]
c : Uye[U-2026-0003, ISIMSIZ, puan=0, STANDART]
Numaralar sinif tarafindan uretildi, disaridan alinmadi.

--- Puan islemleri ---
a : Uye[U-2026-0001, Mert Aydin, puan=120, GUMUS]
```

```
a : Uye[U-2026-0001, Mert Aydin, puan=520, ALTIN]
  RED: yetersiz puan (700 > 520).
a : Uye[U-2026-0001, Mert Aydin, puan=500, ALTIN]

--- Salt-okunur alan denemesi ---
a.getUyeNo() = U-2026-0001  (yalnizca okunabilir)
c : Uye[U-2026-0003, Deniz Ak, puan=0, STANDART]
```

> **BİLGİ**
>
> `private static int sonrakiSira` bildirimindeki `static`, alanın nesnelere değil **sınıfa** ait olduğunu, yani tek kopya bulunduğunu söyler. Bu konunun ayrıntısı Hafta 6'nın işidir; bu görevde yalnızca "tüm nesnelerin paylaştığı bir sayaç" olarak kullanmanız yeterlidir.

> **DİKKAT**
>
> Numara biçimlendirmesi için `String.format("U-%s-%04d", yil, sira)` kullanın. `%04d` biçim belirteci sayıyı dört haneye tamamlar: `1` → `0001`.

Kontrol Soruları

1. `uyeNo` alanını yalnızca `private` yapsaydınız (yani `final` olmasaydı) hangi risk doğardı? Sınıfın **içinden** yazılabilecek bir hata senaryosu yazın.
2. `puan` alanı için setter yazmamak, `puanEkle()`/`puanHarca()` yazmak hangi iş kuralını korumayı mümkün kılıyor?
3. `getSeviye()` metodunun karşılığı olan bir alan tutulsaydı ne olurdu?

Görev 4 — Değişmez (Immutable) Sınıf

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Oluşturulduktan sonra durumu hiçbir biçimde değiştirilemeyen bir sınıf tasarlamak ve dönüşüm metotlarının **yeni nesne** ürettiğini göstermek.

Görev Tanımı

İki boyutlu düzlemde bir noktayı temsil eden `Koordinat` adında **değişmez** bir sınıf yazın. Ders notu Bölüm 4.4'teki dört kuralı eksiksiz uygulayın.

Gereksinimler

1. Sınıf `public final class Koordinat` olarak bildirilmelidir.
2. `x` ve `y` alanları `private final double` olmalıdır.
3. **Hiçbir setter yazılmayacaktır.**
4. `otele(double dx, double dy)` ve `olcekle(double k)` metotları mevcut nesneyi değiştirmeden **yeni bir `Koordinat`** döndürmelidir.
5. `uzaklik(Koordinat digeri)` iki nokta arası öklid uzaklığını döndürür.
6. `public static final Koordinat BASLANGIC` sabiti (0,0) noktasını tutmalıdır.
7. `baslangictanUzaklik()` metodu `uzaklik(BASLANGIC)` çağırmalıdır (türetilmiş).
8. `toString()` (3.00, 4.00) biçiminde yazdırmalıdır.
9. Sürücü programda dönüşümlerin **zincirleme** kullanılabildiğini gösterin: `new Koordinat(1,1).otele(1,2).olcekle(2).otele(-1,0)`

Beklenen Çıktı

```
▸ TERMINAL — java Gorev4_KoordinatDemo
p                 : (3.00, 4.00)
baslangica uzaklik: 5.00

--- Donusumler YENI nesne uretir ---
q = p.otele(2,-1) : (5.00, 3.00)
r = q.olcekle(3)  : (15.00, 9.00)
p (hala)          : (3.00, 4.00)   <- DEGISMEDI
q (hala)          : (5.00, 3.00)   <- DEGISMEDI

--- Zincirleme kullanim ---
```

```
z                : (3.00, 6.00)
uzaklik(p, z)    : 2.0000

--- Paylasim guvenli ---
  metot icinde uretilen: (100.00, 100.00)
BASLANGIC (hala)  : (0.00, 0.00)
```

İPUCU

Zincirleme çağrının çalışmasının nedeni, her dönüşüm metodunun `Koordinat` türünde bir nesne döndürmesidir. Dönen nesne üzerinde hemen bir sonraki metot çağrılabilir. Bu, `String` sınıfında her gün kullandığınız `s.trim().toUpperCase()` deseninin aynısıdır.

DİKKAT

`bozmayaCalis(Koordinat k)` metodunu yazarken dikkat edin: metot içinde nesneyi değiştirmeye çalışsanız bile **değiştiremezsiniz**, çünkü setter yoktur. Çıktıda `BASLANGIC` sabitinin değişmediğini gösterin.

Kontrol Soruları

1. Sınıfı `final` yapmasaydık, bir alt sınıf değişmezliği nasıl delebilirdi?
2. `otele()` metodu `void` olsaydı ve `this.x += dx;` yazsaydık hangi kural ihlal edilirdi? Bu durumda `BASLANGIC` sabiti ne olurdu?
3. Değişmez nesnenin “paylaşımı güvenlidir” ifadesi ne anlama gelir? Somut bir senaryo yazın.

Görev 5 — İleri Düzey: Paket Düzeni ve Paket-Özel Erişim (Bonus)

Süre	Puan	Zorluk
25 dakika	20 puan (bonus)	●●●

Amaç

Varsayılan (paket-özel) erişimi **bilinçli bir tasarım aracı** olarak kullanmak: iki sınıfın paket içinde yakın işbirliği yapması, ama dışarıya yalnızca denetimli bir yüz göstermesi.

Görev Tanımı

`kutuphane` adında bir paket oluşturun ve içine `Kitap` ile `Katalog` sınıflarını yerleştirin. Ardından **paket dışında** bulunan `Gorev5_KutuphaneDemo` sınıfını yazın.

Gereksinimler

Öğe	Beklenen tasarım
Klasör düzeni	`src/kutuphane/Kitap.java`, `src/kutuphane/Katalog.java`, `src/Gorev5_KutuphaneDemo.java`
`Kitap.isbn`, `baslik`, `yazar`	`private final` · yalnızca getter
`Kitap.oduncte`	**Varsayılan (paket-özel) erişim** — belirteç yazılmaz
`Kitap.isOduncte()`	`public` getter — dışarıya yalnızca **okuma**
`Katalog`	Sabit boyutlu `Kitap[]` dizisi tutar; `ekle()`, `bul()`, `oduncVer()`, `iadeAl()`, `getRaftakiSayisi()`, `listele()`
`Katalog.oduncVer()`	`k.oduncte` alanına **doğrudan** erişir (aynı paket)
`Gorev5_KutuphaneDemo`	`import kutuphane.*;` · `oduncte` alanına **erişemez**, yalnızca `isOduncte()` görür

Beklenen Çıktı

```
▸ TERMINAL — java Gorev5_KutuphaneDemo
=== Baslangic katalogu ===
  978-0 - Java: How to Program (Deitel) [RAFTA]
  978-1 - Core Java Vol. I (Horstmann) [RAFTA]
  978-2 - Effective Java (Bloch) [RAFTA]

=== Odunc islemleri ===
```

```
  OK : Core Java Vol. I odunc verildi.
  RED: Core Java Vol. I zaten oduncte.
  RED: 978-9 katalogda yok.
  RED: Effective Java zaten rafta.

=== Son durum ===
  978-0 - Java: How to Program (Deitel) [RAFTA]
  978-1 - Core Java Vol. I (Horstmann) [ODUNCTE]
  978-2 - Effective Java (Bloch) [RAFTA]
Raftaki kitap sayisi: 2

=== Paket disindan erisim sinirlari ===
a.isOduncte() = false   (public getter)
a.oduncte alanina dogrudan yazma yolu KAPALI.
```

İPUCU

Paketli kod derlerken kaynak kök klasöründen çalışın:
`javac -d bin kutuphane/*.java Gorev5_KutuphaneDemo.java`
Çalıştırırken sınıf yolunu `bin` olarak verin: `java -cp bin Gorev5_KutuphaneDemo`

DİKKAT

Paket bildirimi (`package kutuphane;`) dosyanın **ilk kod satırı** olmalıdır; yalnızca yorumlar ondan önce gelebilir. Klasör adı ile paket adı birebir aynı olmak zorundadır.

Kontrol Soruları

1. `oduncte` alanını `private` yapsaydık `Katalog` sınıfı nasıl çalışırdı? Bu tasarım hangi bakımdan daha iyi, hangi bakımdan daha kötü olurdu?
2. `oduncte` alanını `public` yapsaydık hangi iş kuralı delinirdi? Somut bir kod satırıyla gösterin.
3. Paket-özel erişim ile `protected` arasındaki fark bu görevde nerede kendini gösterirdi?

Değerlendirme ve Teslim

Puanlama Ölçütleri

Görev	Ölçüt	Puan
Görev 1	Tüm alanlar `private`, `barkod` `final` (5) · doğrulamalı setter'lar (5) · `stokGir`/`stokCik` iş metotları (6) · türetilmiş `getStokDegeri()` (4)	20
Görev 2	`ayarla()` tek metotta iki alan (8) · aralık ve büyüklük denetimi (6) · ret durumunda hiçbir alan değişmiyor (4) · `toString()` (2)	20
Görev 3	`uyeNo` `private final`, setter yok (6) · numara sınıfça üretiliyor (6) · `puanEkle`/`puanHarca` kuralları (5) · türetilmiş `getSeviye()` (3)	20
Görev 4	`final class` + `private final` alanlar (6) · setter yok (4) · dönüşümler yeni nesne döndürüyor (6) · zincirleme çağrı gösterimi (4)	20
Görev 5	Paket düzeni doğru (6) · `oduncte` paket-özel (6) · `Katalog` paket içi erişimi kullanıyor (4) · demo yalnızca `public` yüzü görüyor (4)	20 (bonus)
Toplam	Görev 1–4 zorunlu · Görev 5 bonus	**80 + 20**

Aşağıdaki durumlarda görev puanından kesinti uygulanır:

Durum	Kesinti
Alan `public` bırakılmış	Görev puanının %50'si
Setter yazılmış ama doğrulama yok	Görev puanının %40'ı
Geçersiz istekte alan yine de değişiyor	Görev puanının %40'ı
`toString()` ezilmemiş	2 puan
Türkçe karakter içeren tanımlayıcı	3 puan
Dosya adı ile sınıf adı uyuşmuyor / derlenmiyor	Görev puanının tamamı

Teslim Öncesi Kontrol Listesi

Teslim Öncesi Kontrol Listesi

Tüm dosyalar hatasız derleniyor (`javac -d bin src/*.java` uyarı vermiyor).

Her sınıfın **tüm alanları** `private`; tek istisna `public static final` sabitler.

Yazdığım her setter'ın içinde bir doğrulama koşulu var.

Geçersiz istek gönderdiğimde nesne eski geçerli durumunu koruyor.

Salt-okunur olması gereken alanlar `final` ve setter'ları yok (`barkod`, `uyeNo`, `x`, `y`).

`Koordinat` sınıfım `final`; hiç setter içermiyor; dönüşümler yeni nesne döndürüyor.

Türetilebilen hiçbir bilgi için ayrı alan tutmadım (`stokDegeri`, `seviye`, `nabizBasinci`).

Her sınıfta `toString()` metodunu ezdim.

Tanımlayıcılarda Türkçe karakter kullanmadım; adlandırma kurallarına uydum.

Dosya adları içindeki `public` sınıf adlarıyla birebir aynı; arşiv adı `Hafta04_<Numara>_<AdSoyad>.zip` biçiminde.

Sık Karşılaşılan Hatalar

Belirti	Olası neden	Çözüm
`The field Urun.fiyat is not visible`	Alan `private`, dışarıdan erişiliyor	Getter/setter kullanın
`The final field Koordinat.x cannot be assigned`	`final` alana ikinci kez atama	Yeni nesne döndürün
`The blank final field uyeNo may not have been initialized`	Bazı yapıcı yollarında atama yok	Tüm yollarda alana değer atayın
`The type kutuphane.Kitap is not visible`	Sınıf `public` değil	`public class Kitap` yazın
`The public type Katalog must be defined in its own file`	Dosya adı sınıf adıyla uyuşmuyor	Dosyayı `Katalog.java` yapın
Değer değişiyor ama uyarı yazılmıyor	Setter'da `return;` unutulmuş	Uyarıdan sonra `return;` ekleyin
`Exception in thread "main" ... NullPointerException`	`ad` alanı hiç atanmamış	Yapıcıda başlangıç değeri verin

> **BİLGİ**
>
> Laboratuvar sonunda çözümler paylaşılacaktır. Ancak çözümü görmeden önce görevleri kendiniz tamamlamanız, Hafta 5'teki yapıcı konusunu izleyebilmeniz için önemlidir; bu haftanın setter düzeni orada yapıcı düzenine dönüşecektir.